

Laboratorio di architettura degli elaboratori

Circuiti logici combinatori

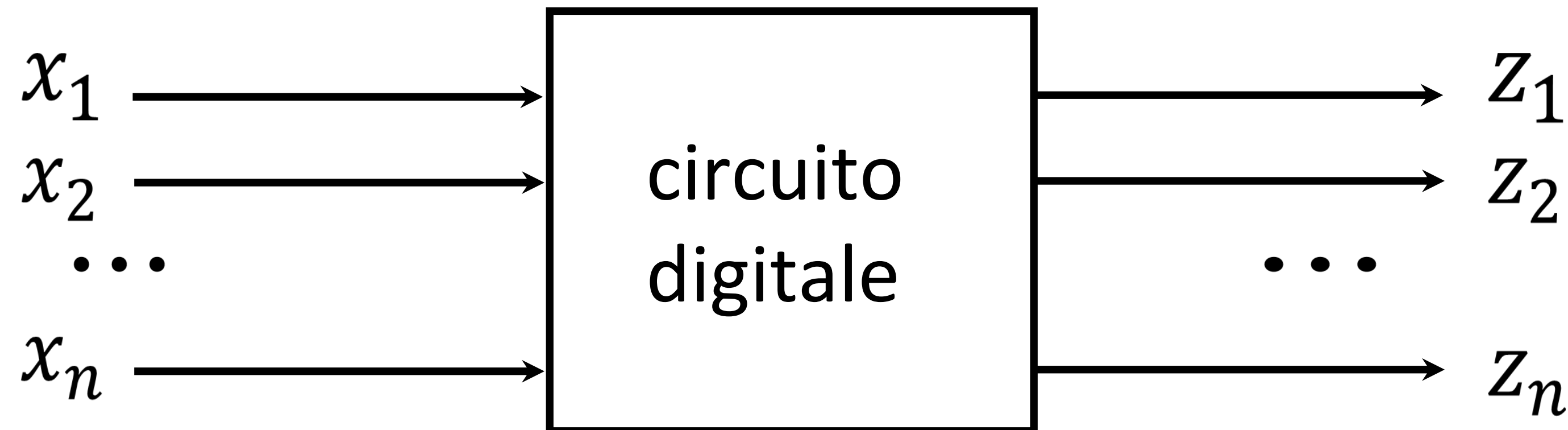
Obiettivi

- Imparare ad usare il simulatore online CircuitVerse
- Sperimentare i circuiti logici elementari in modo strutturato

Circuito combinatorio

Output è **funzione esclusivamente dell'input**, quindi ogni output z_i è una funzione booleana degli ingressi $x_1, \dots, x_n$

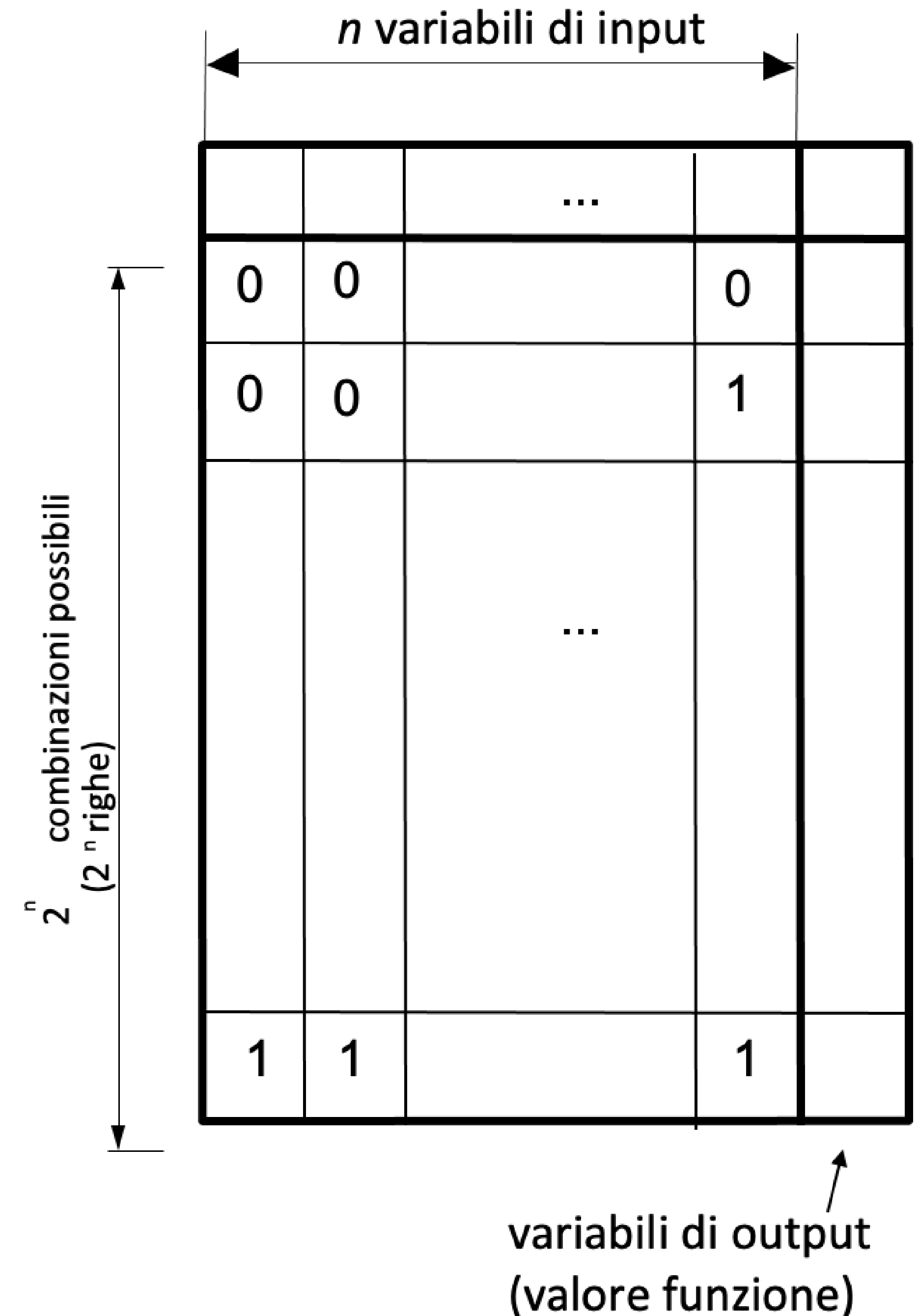
$$z_i = f_i(x_1, x_2, \dots, x_n)$$



Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A}\bar{B}$

Algebra booleana

- **Tabelle di verità:** descrivono completamente il valore di una funzione Booleana attraverso tutte le combinazioni di input; n input corrispondono a 2^n combinazioni (righe)
- **Forma canonica:** righe ordinate per valori crescenti degli ingressi interpretando i valori delle variabili di ingresso come cifre di una codifica binaria



Forme canoniche

- **Formula normale disgiuntiva (FND):**
 - **Sommatoria (OR) di termini** ciascuno dei quali è una produttoria (AND) di letterali costituiti da nomi di variabili di ingresso o da negazioni dei nomi di variabili di ingresso.
- **Formula normale congiuntiva (FNC):**
 - Concetto duale del precedente ossia è una **produttoria (AND) di termini** ciascuno dei quali è una sommatoria (OR) di letterali costituiti da nomi di variabili di ingresso o da negazioni di nomi di variabili di ingresso

Funzione di maggioranza

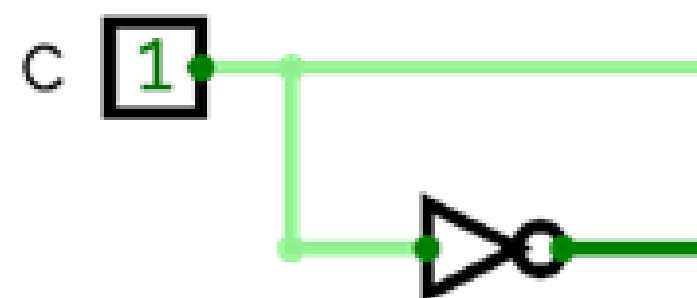
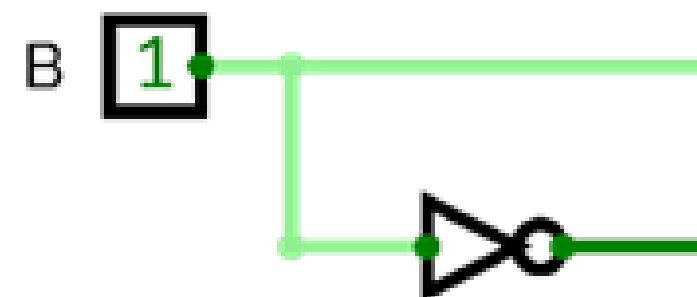
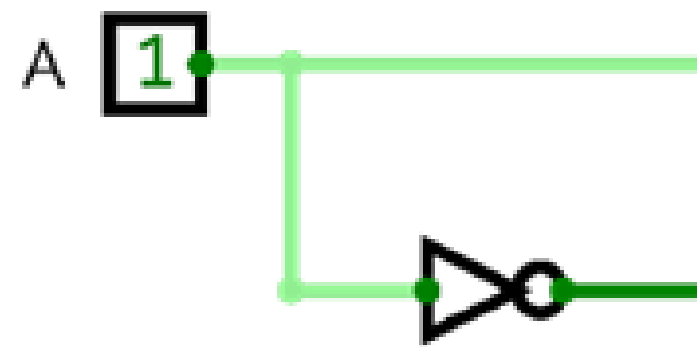
- Funzione di maggioranza su tre input: restituisce 1 se la maggioranza degli input è 1; 0 altrimenti
- M (l'output) vale 1 nelle righe
 - $\overline{A}BC$;
 - $A\overline{B}C$;
 - $AB\overline{C}$;
 - ABC
- quindi la funzione M è, in **forma normale disgiuntiva**
$$M = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$
 - mettendo in OR anche i casi dove M vale 0 non cambierebbe nulla

input			output
A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Dalla tabella di verità alla formula algebrica

1. Scrivere la tabella di verità per la funzione
2. Disporre gli invertitori per generare il complemento di ogni input

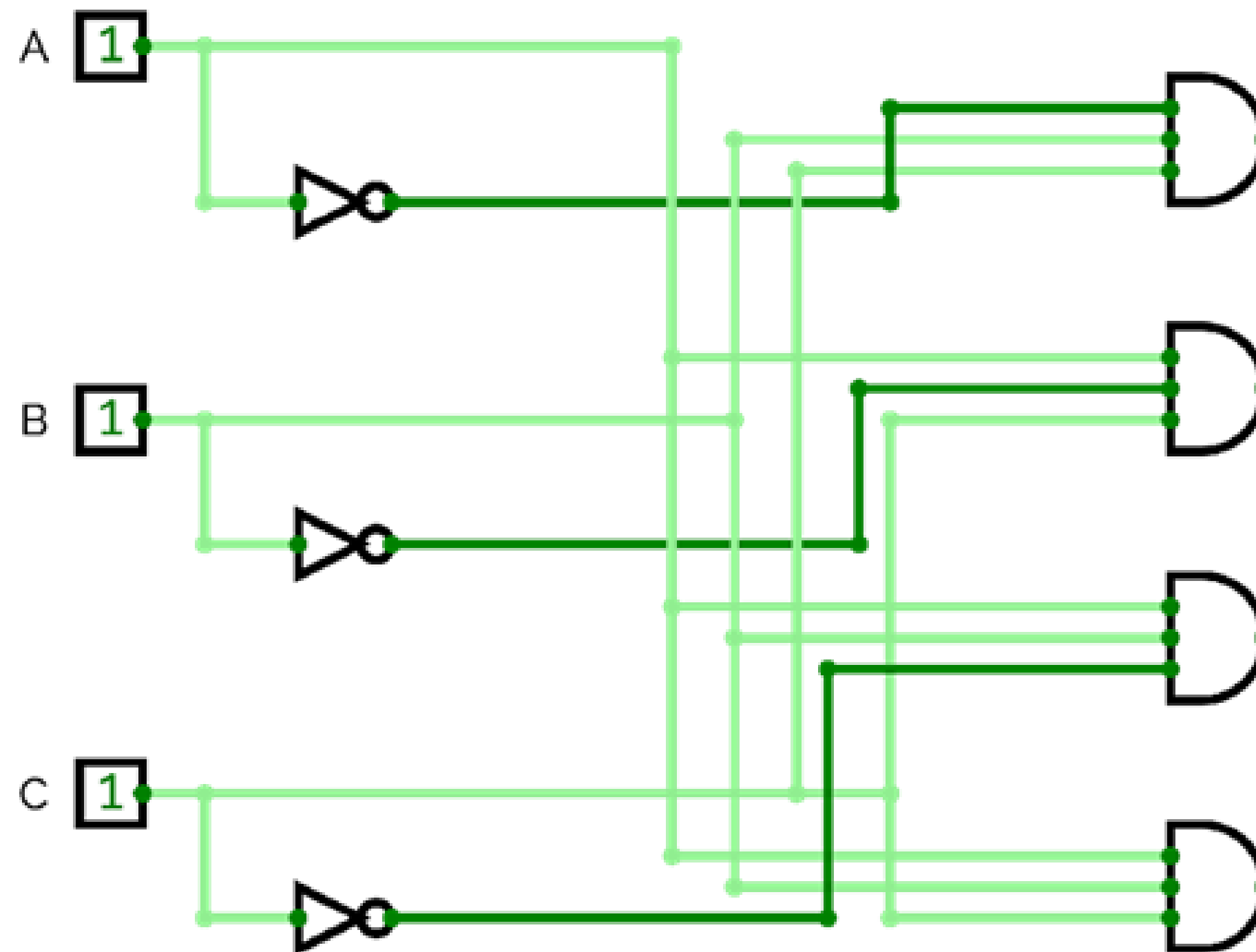
$$M = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$



Dalla tabella di verità alla formula algebrica

1. Scrivere la tabella di verità per la funzione
2. Disporre gli invertitori per generare il complemento di ogni input
3. Introdurre una porta AND per ogni termine con un 1 nella colonna dei risultati
4. Collegare le porte AND agli input appropriati

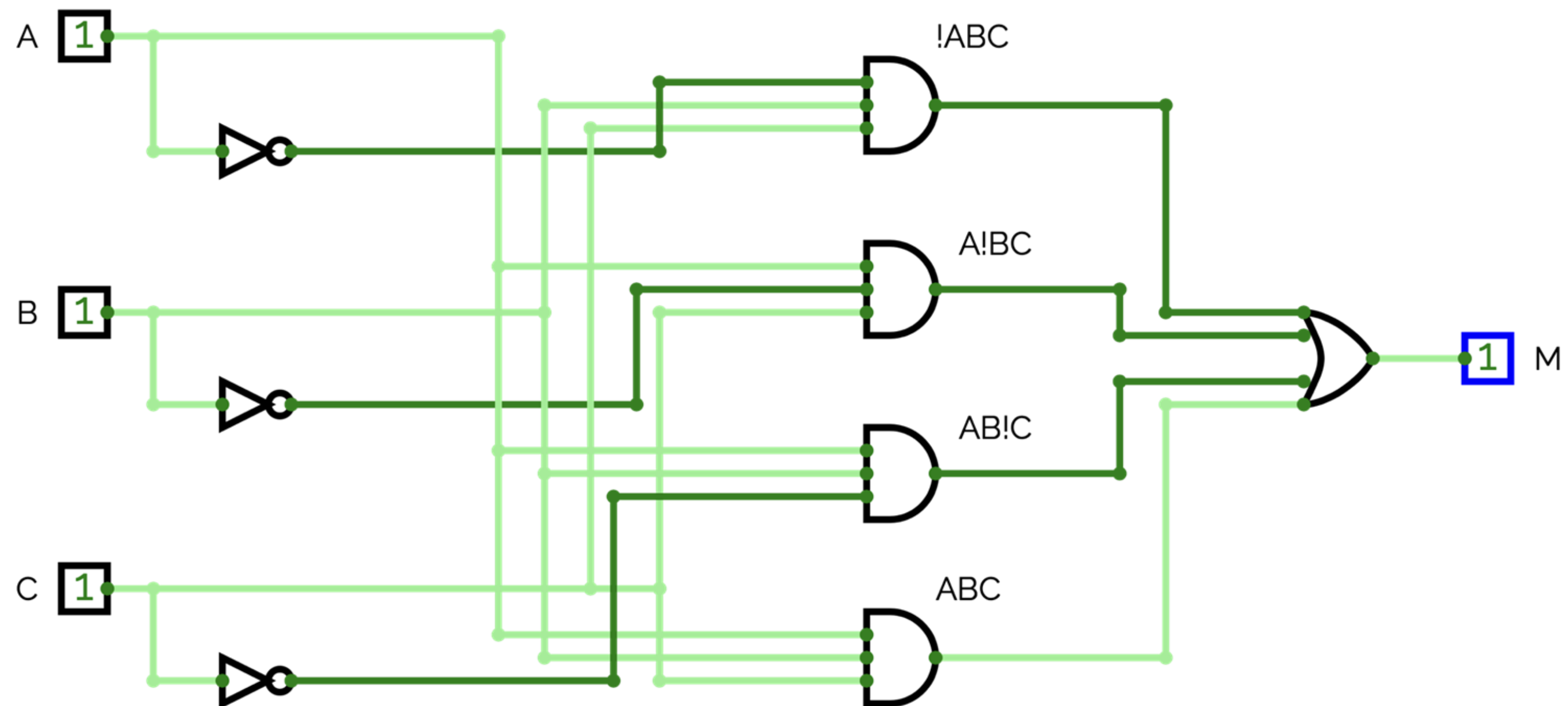
$$M = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$



Dalla tabella di verità alla formula algebrica

1. Scrivere la tabella di verità per la funzione
2. Disporre gli invertitori per generare il complemento di ogni input
3. Introdurre una porta AND per ogni termine con un 1 nella colonna dei risultati
4. Collegare le porte AND agli input appropriati
5. Inviare l'output di tutte le porte AND in una porta OR

$$M = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$



Esercizi

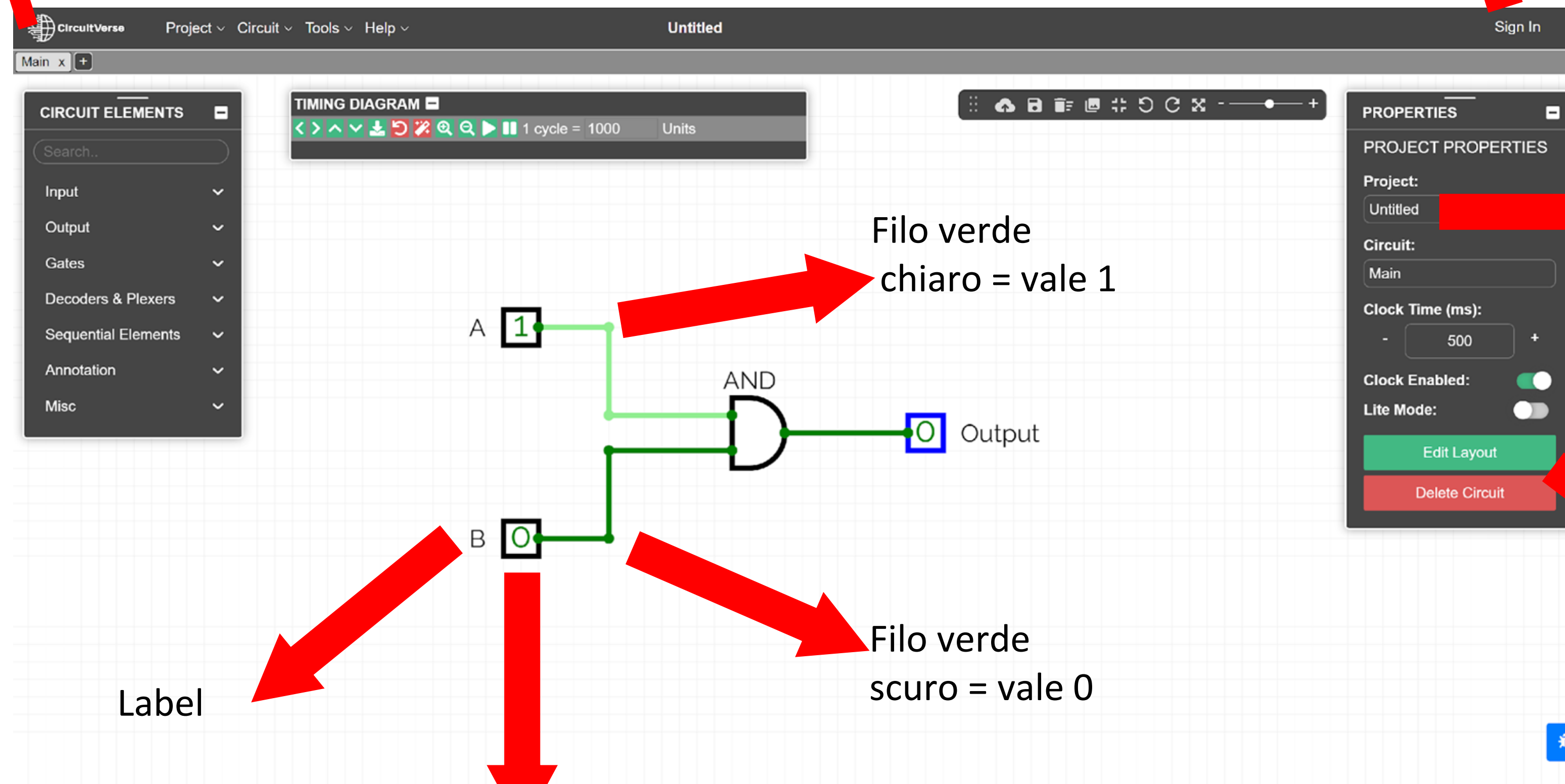
Simulatore CircuitVerse

- Cloud-based, realizzato in HTML5:
 - <https://circuitverse.org>
 - è necessario registrarsi con email/password per salvare i propri progetti e per accedere a quelli condivisi

Simulatore CircuitVerse

Registrarsi con email/password per accedere
poi alla soluzione degli esercizi

Il **progetto** è costituito da vari **circuiti**,
che possono essere riutilizzati all'interno di altri circuiti



Label

Filo verde
chiaro = vale 1

Filo verde
scuro = vale 0

Date un nome al
progetto

Bottone per
modificare il layout

Input binario: 0/1 (si cambia cliccandoci sopra)

Esercizio 1: half_adder

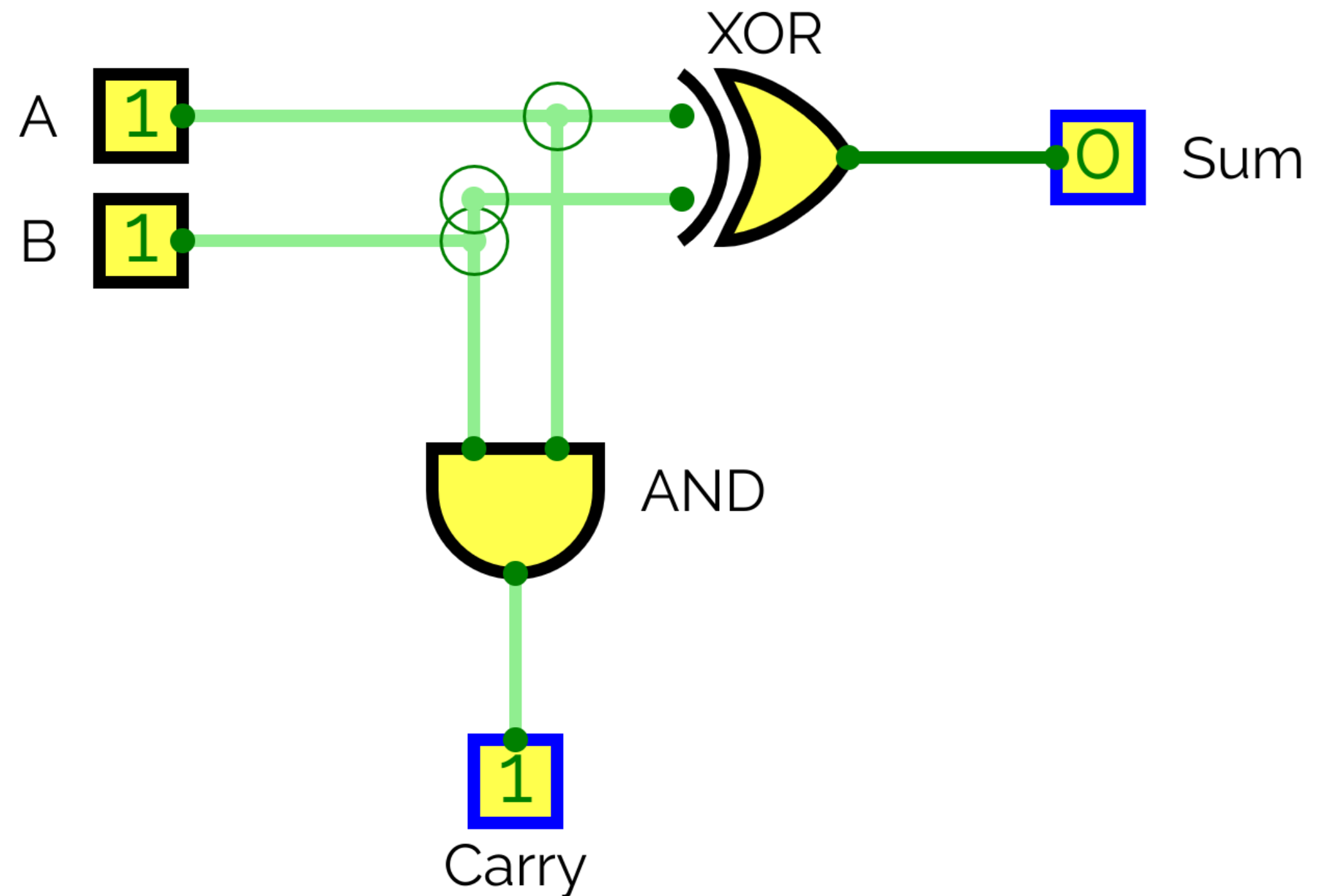
ingressi		uscite	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Costruire il circuito corrispondente all'half adder descritto in tabella.

Esercizio 1: half_adder

ingressi		uscite	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Costruire il circuito corrispondente all'half adder descritto in tabella.



Esercizio 2: full_adder

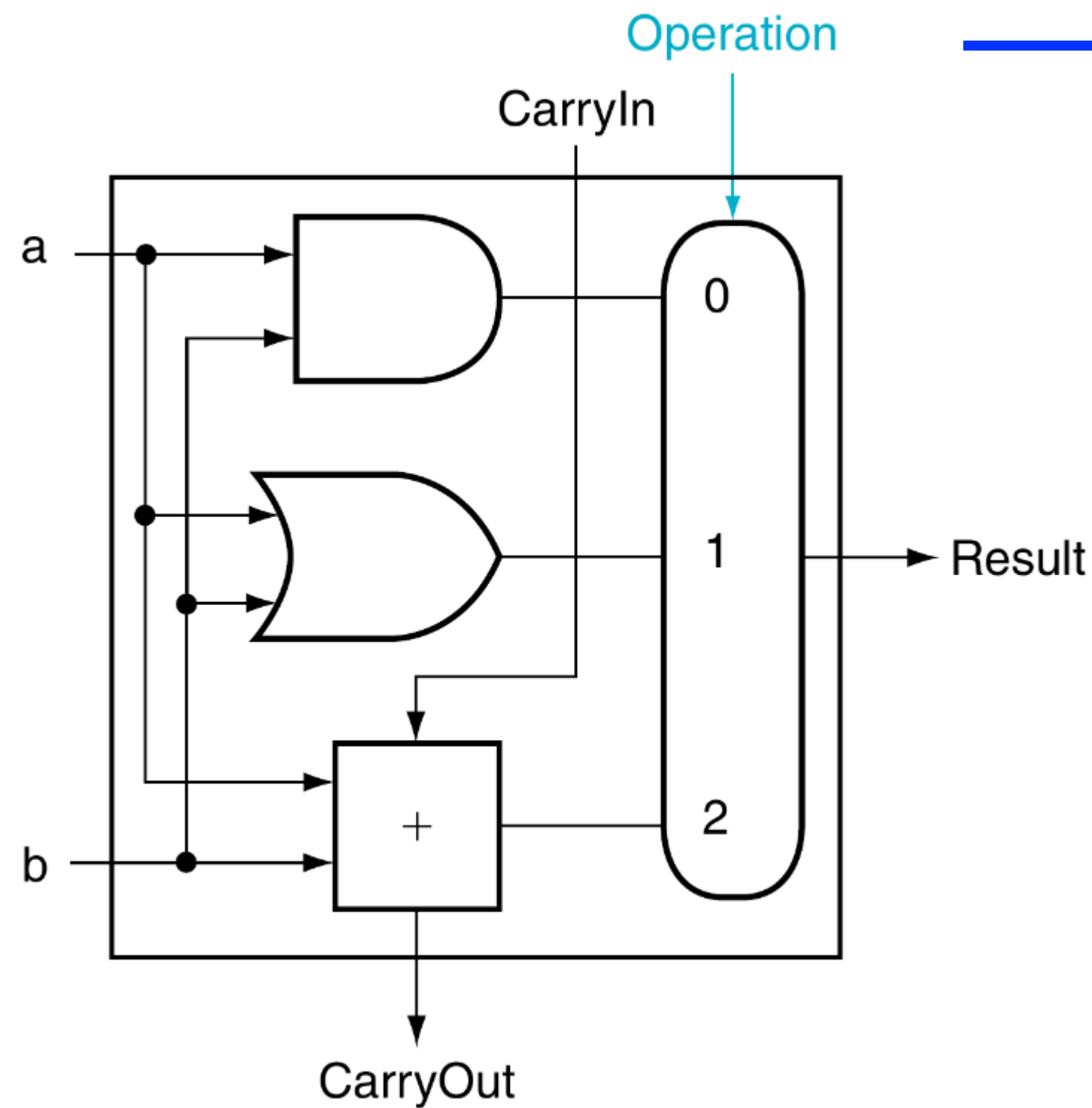
Costruzione del full adder, utilizzando l'half adder dell'esercizio precedente.

nuovo input


Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

FIGURE A.5.3 Input and output specification for a 1-bit adder.

Esercizio 3: ALU a 1 bit (ALU_1bit_v0)



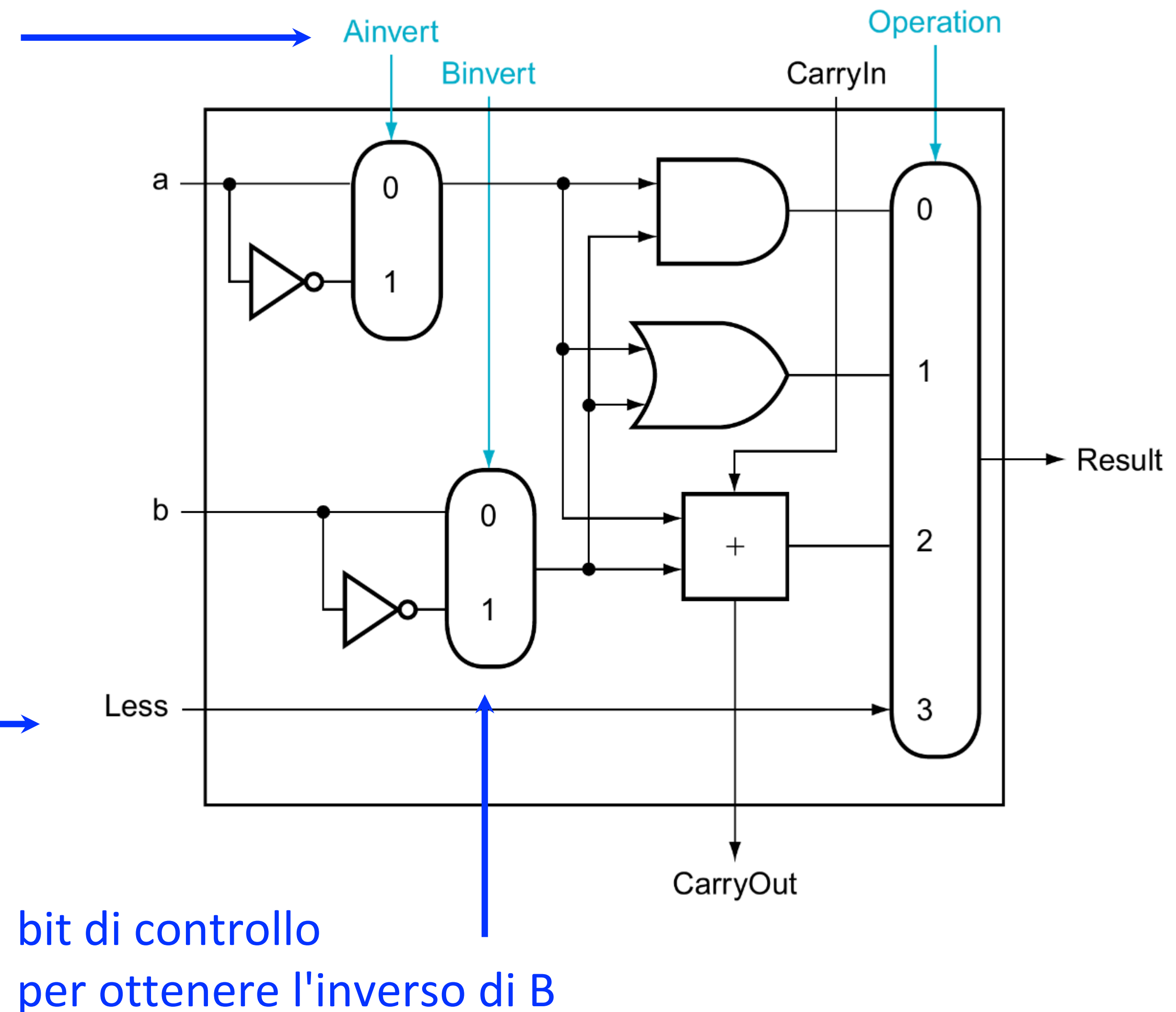
Segnale di controllo
codificato su 2 bit, che
indica l'operazione:

- 00: AND logico
- 01: OR logico
- 10: somma aritmetica completa
- 11: non usato

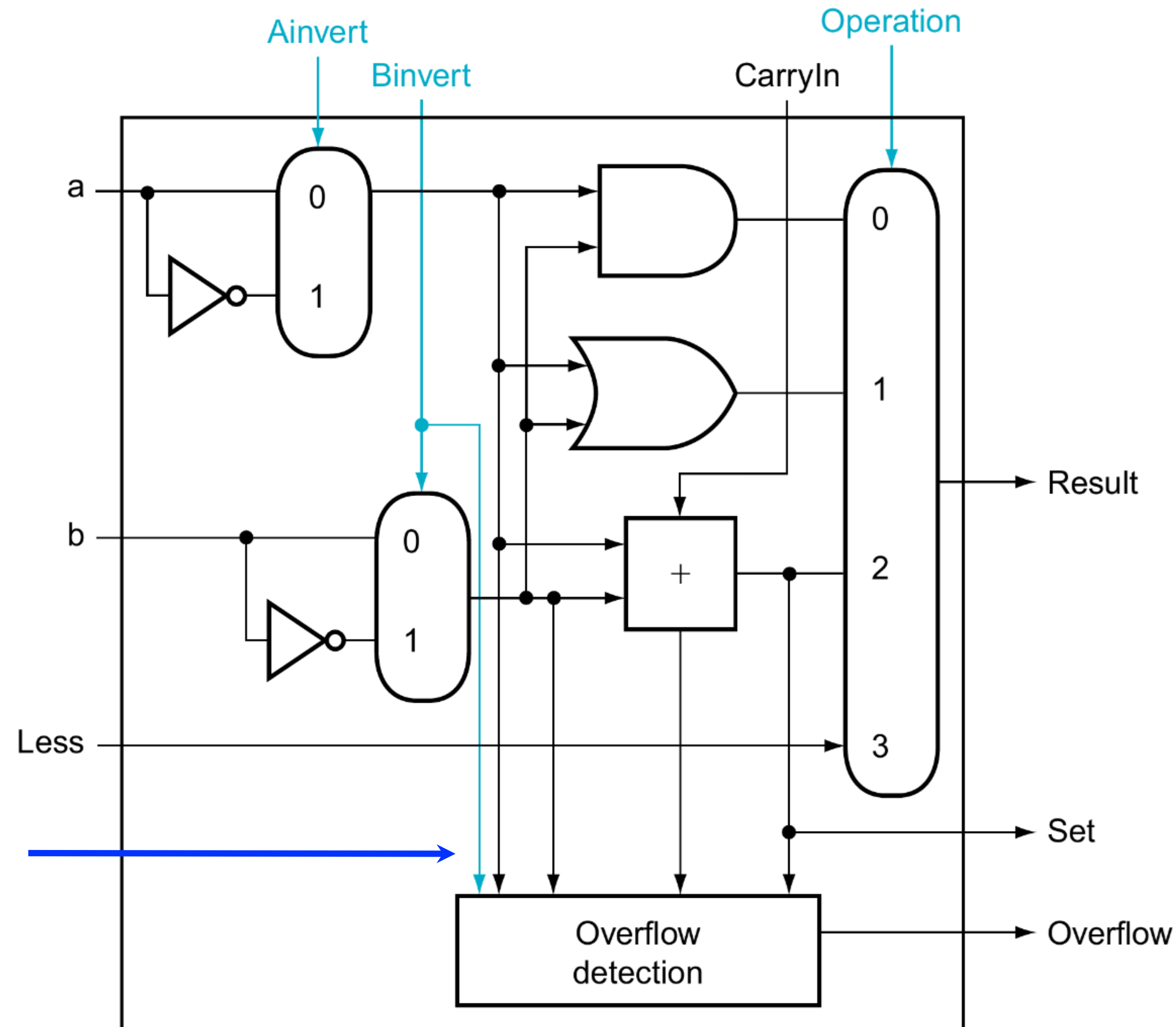
Esercizio 4: ALU a 1 bit (ALU_1bit_v1)

bit di controllo
per ottenere l'inverso di A
(suggerimento: xor)

bit ulteriore di input
collegato al caso 11 del
multiplexer



Esercizio 5: ALU a 1 bit MSB (ALU_1bit_v2)



Sono necessari tutti
questi ingressi?

NO!

Es 6: ALU RISC-V a 4 bit

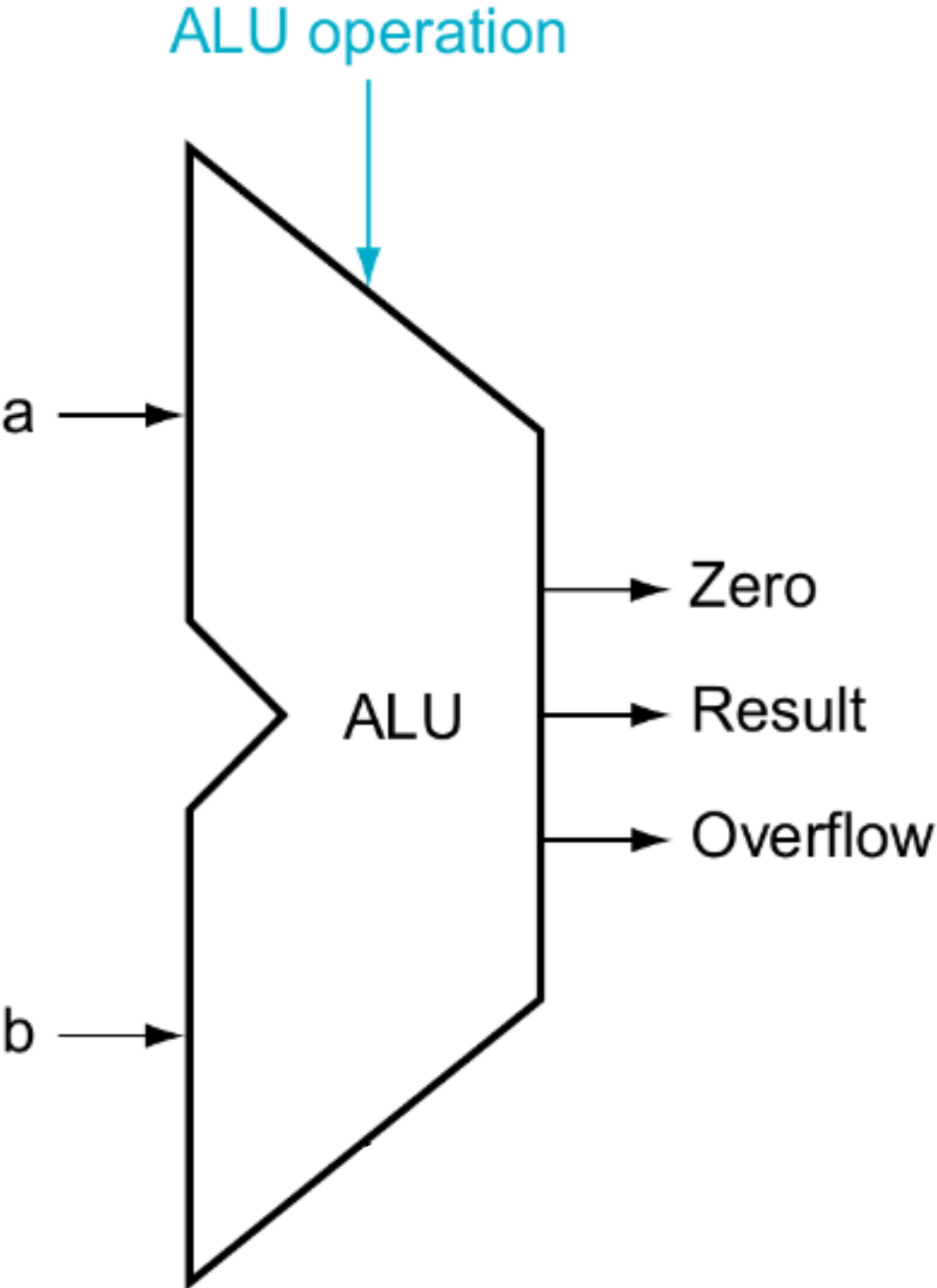
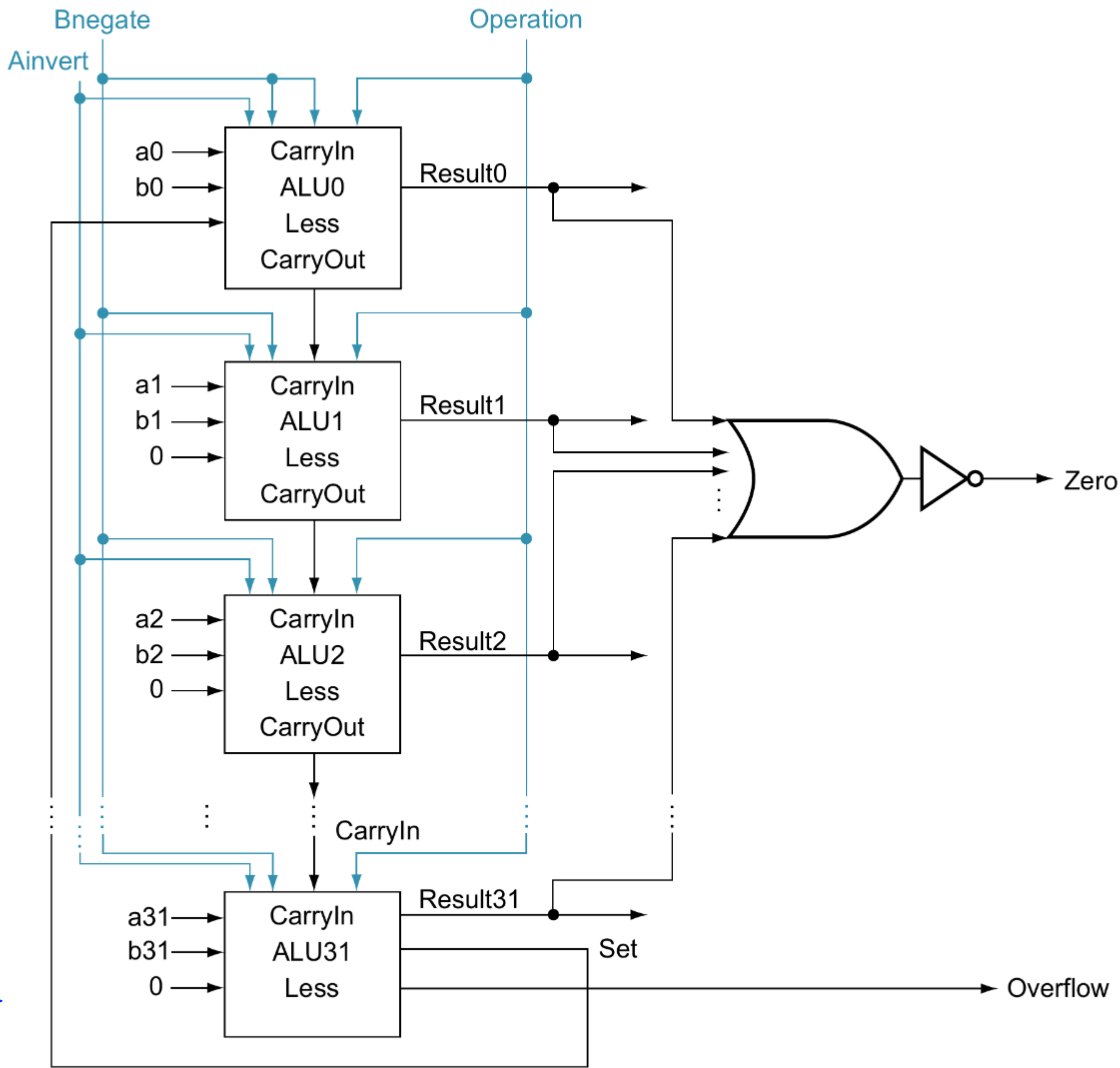
(ALU_4bit)

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

- Mettendo in cascata 4 ALU a 1 bit come la precedente, pensare a come costruire una ALU completa a 4 bit in grado di fare le seguenti operazioni su ingressi A e B a 4 bit:
 - AND
 - OR
 - add
 - subtract
 - set less than (restituisce 1 se $A < B$; 0 altrimenti)
 - NOR
- aggiungere anche l'uscita zero (che vale 1 se il risultato è zero) e l'uscita che segnala overflow.

Es 6: ALU RISC-V a 4 bit

(ALU_4bit)

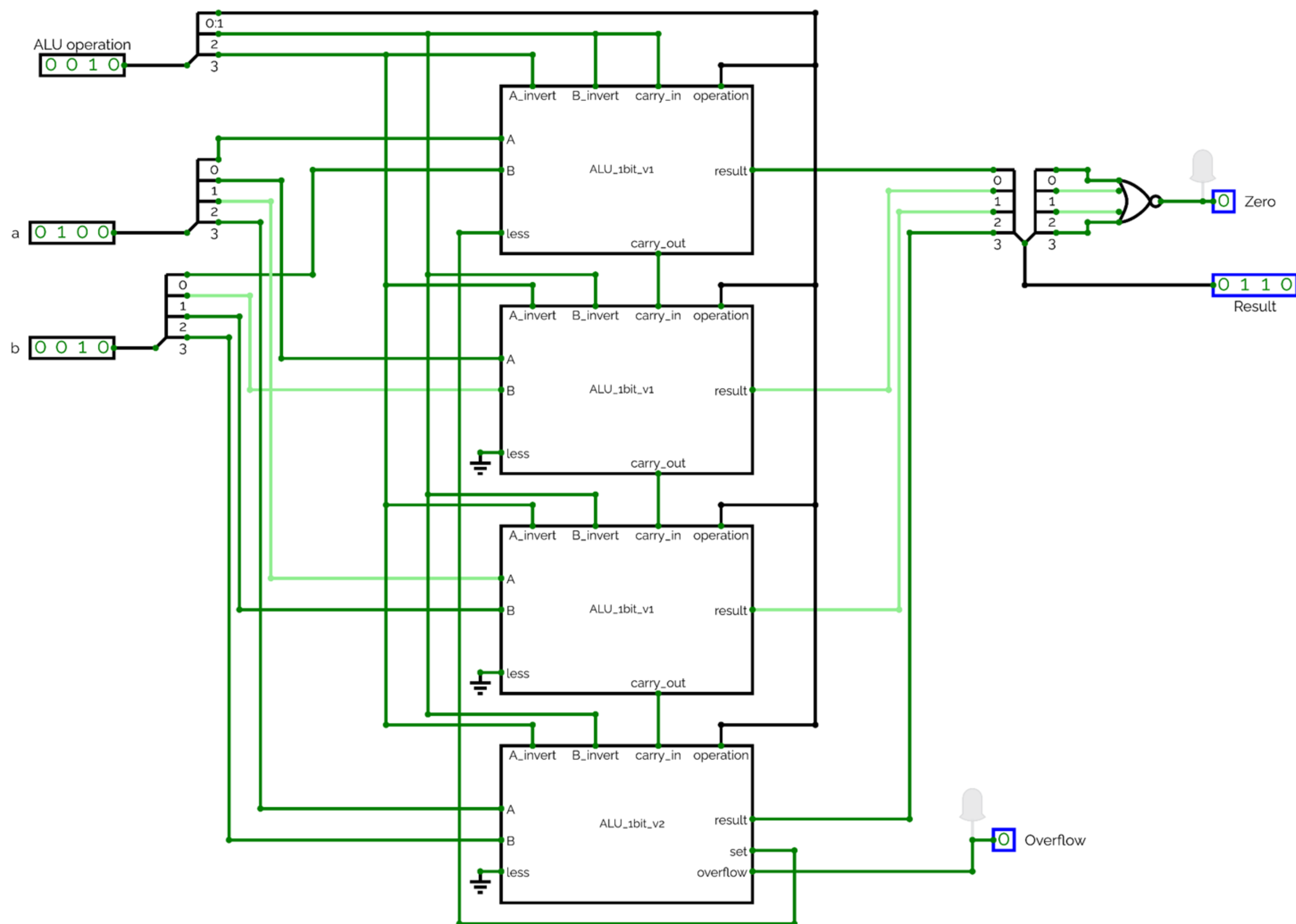


esempio a 32bit



Es 6: ALU RISC-V a 4 bit

(ALU_4bit)



Esercizio 7

"Sanity check":

- 1. Dato $A = 1101$, "estrarre" i suoi 2 bit meno significativi
- 2. Dati $A = 5$, $B = 3$, fare la somma $A + B$
- 3. Dati $A = 5$, $B = -3$, fare la somma $A + B$
- 4. Dati $A = -5$, $B = 3$, fare la sottrazione $A - B$
- 5. Per generici A e B , calcolare se $A=B$
- 6. Calcolare i 4 possibili risultati di $X \text{ NOR } Y$

Il «set less than» è completo come mostrato nel testo?

- 1. Con $A = 3$, $B = 5$, stabilire con «set less than» se $A < B$
- 2. Con $A = 3$, $B = -5$, stabilire con «set less than» se $A < B$. Cosa succede?
- 3. Con $A = -5$, $B = 5$, stabilire se $A < B$. Cosa succede?

Se qualcosa non vi torna ...

- 1. Modificare la ALU per gestire correttamente «set less than»

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR